CI/CD Pipeline Project

Complete Interview Preparation Guide

GitHub → Jenkins → Maven → Nexus → Tomcat

DEVOPS LEARNING SERIES | SESSIONS 1 - 6

AWS EC2 | Ubuntu 22.04 | t3.small | Java 17
Jenkins :8080 | Nexus :8081 | Tomcat :8082

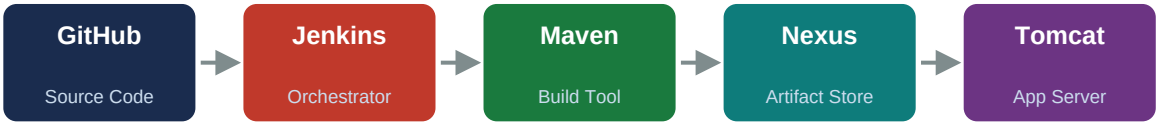
Maven Build • Artifact Management • Servlet Deployment • Pipeline Automation

01 The Big Picture

What you built and how it fits together

Pipeline Architecture

Every CI/CD pipeline follows the same shape: code lives in a version control system, a build server detects changes and compiles the code, the artifact is stored in a repository, and finally deployed to a runtime. The tools change; the shape never does.



Infrastructure

Component	Detail	Why
AWS EC2	t3.small (2GB RAM, 2 vCPU)	Runs Jenkins + Nexus + Tomcat
OS	Ubuntu 22.04 LTS	Industry standard, excellent tooling support
Storage	20 GB gp3 EBS	Block storage — persistent across instance stops
Java	OpenJDK 17 (JDK)	Required by Maven (build) and Tomcat (run)
Network	Security Group: ports 22, 8080, 8081, 8082	SSH + service access

---

Port Map — the three services on one server

Service	Port	Access URL	Why this port
Jenkins	8080	http://<EC2-IP>:8080	Jenkins default — kept as-is
Nexus	8081	http://<EC2-IP>:8081	Nexus default — kept as-is
Tomcat	8082	http://<EC2-IP>:8082	Changed from 8080 to avoid Jenkins conflict

02 Session Breakdown

What each session covered

Session 1: Maven + Java Web App

Installed Java 17 + Maven on EC2. Created a Java servlet (HelloServlet.java). Understood pom.xml (groupId, artifactId, version, packaging, dependencies, scope). Ran mvn clean package and produced hello-app-1.0.0.war. Explored target/ contents.

Session 2: Tomcat — Deploy your .war

Installed Tomcat 9 on EC2 at port 8082. Changed default port in conf/server.xml. Learned Tomcat folder structure (bin/, conf/, webapps/, logs/). Manually deployed .war by copying to webapps/. Accessed live app in browser at http://:8082/hello-app-1.0.1/hello.

Session 3: Nexus — Artifact Repository

Installed Nexus 3 on EC2 at port 8081. Created nexus system user. Understood repositories: maven-releases, maven-snapshots, maven-public. Configured pom.xml distributionManagement and ~/.m2/settings.xml credentials. Ran mvn clean deploy to push .war to Nexus with version history.

Session 4: Jenkins — Checkout + Build Stages

Installed Jenkins as a systemctl service at port 8080. Ran initial setup, installed suggested plugins. Configured Maven in Global Tools. Pushed hello-app code to GitHub. Added Jenkinsfile to repo. Created Pipeline job pointing at GitHub. First successful build: Checkout + Build stages.

Session 5: Full Pipeline — Nexus + Tomcat Stages

Extended Jenkinsfile with two more stages: Push to Nexus (mvn deploy) and Deploy to Tomcat (Deploy to container plugin). Configured Jenkins credentials for Nexus and Tomcat. Complete pipeline: GitHub -> Jenkins -> Maven -> Nexus -> Tomcat.

Session 6: Webhook — Zero-Touch Automation

Configured GitHub webhook pointing to http://:8080/github-webhook/. Every git push now triggers the full pipeline automatically. No manual Build Now required. Tested end-to-end: push code, wait 60 seconds, see new version live on Tomcat.

03 Tool Deep Dives

Key facts, files, and commands for each tool

MAVEN

What it is: A build automation tool for Java. Reads pom.xml to compile code, run tests, download dependencies, and package into a .jar or .war.

Pipeline role: Compiles Java source -> runs tests -> produces the deployable .war artifact.

Key paths: pom.xml (project config) | target/ (build output) | ~/.m2/ (local cache) | ~/.m2/settings.xml (server credentials)

Essential commands:

```
mvn clean package # clean old output, compile, test, package mvn clean deploy # everything above + push artifact to Nexus mvn compile # compile only (no packaging) mvn test # run tests only mvn -version # confirm Maven is installed
```

TOMCAT

What it is: A servlet container (runtime environment) for Java web applications. Listens for HTTP requests, loads .war files, and routes requests to servlet classes.

Pipeline role: Receives the .war artifact and serves it as a live web application over HTTP.

Key paths: /opt/tomcat9/bin/ (startup.sh, shutdown.sh) | /opt/tomcat9/conf/server.xml (ports) | /opt/tomcat9/webapps/ (drop .war here to deploy) | /opt/tomcat9/logs/catalina.out (debug)

Essential commands:

```
sudo /opt/tomcat9/bin/startup.sh # start Tomcat sudo /opt/tomcat9/bin/shutdown.sh # stop Tomcat tail -f /opt/tomcat9/logs/catalina.out # watch live logs ls /opt/tomcat9/webapps/ # see deployed apps sudo cp app.war /opt/tomcat9/webapps/ # manual deploy
```

NEXUS

What it is: An artifact repository manager. Stores every built .war/.jar with version numbers permanently. Acts as the single source of truth for "what was built and when".

Pipeline role: Receives the built .war from Jenkins/Maven and stores it versioned for deployment or rollback.

Key paths: /opt/nexus/ (binaries) | /opt/sonatype-work/ (artifact data + DB) | /opt/sonatype-work/nexus3/log/nexus.log (debug) | UI at http://:8081 | Repos: maven-releases, maven-snapshots

Essential commands:

```
sudo -u nexus /opt/nexus/bin/nexus start # start Nexus sudo -u nexus
/opt/nexus/bin/nexus stop # stop Nexus sudo -u nexus /opt/nexus/bin/nexus status #
check status tail -f /opt/sonatype-work/nexus3/log/nexus.log # watch logs cat
/opt/sonatype-work/nexus3/admin.password # get default password
```

JENKINS

What it is: A CI/CD automation server. Reads a Jenkinsfile from your Git repo and runs the pipeline stages automatically when code is pushed.

Pipeline role: Orchestrates the entire pipeline: detects GitHub push, runs Maven build, pushes to Nexus, deploys to Tomcat.

Key paths: /var/lib/jenkins/ (Jenkins home) | /var/lib/jenkins/workspace/ (cloned repos + builds) | /var/lib/jenkins/jobs/ (job configs) | /var/log/jenkins/jenkins.log (debug)

Essential commands:

```
sudo systemctl start jenkins # start Jenkins service sudo systemctl stop jenkins #
stop Jenkins sudo systemctl status jenkins # check running status sudo systemctl
enable jenkins # auto-start on reboot sudo cat
/var/lib/jenkins/secrets/initialAdminPassword # first login
```

04 The Jenkinsfile Decoded

Every line explained

The Jenkinsfile lives in your GitHub repository root. It defines the entire pipeline as code — versioned, reviewable, and reproducible.

Line / Block	What it means
`pipeline { }`	Declares this is a Declarative Pipeline (Jenkins syntax)
`agent any`	Run on any available build agent. Our only agent is the Jenkins server itself.
`tools { maven 'Maven' }`	Use the Maven installation named "Maven" configured in Jenkins Global Tools. Jenkins handles PATH automatically.
`stages { }`	Container for all stages. Stages run sequentially. If one fails, subsequent stages are skipped.
`stage('Checkout') { }`	First stage: pull source code from GitHub into the Jenkins workspace.
`checkout scm`	Tells Jenkins to clone/pull from the SCM (GitHub) configured in the job. Automatic — no hardcoded URL needed.
`stage('Build') { }`	Second stage: compile the code and package into .war.
`sh 'mvn clean package'`	Runs a shell command on the build agent. Same command you ran manually in Session 1.
`stage('Push to Nexus') { }`	Third stage: upload the built .war to Nexus for permanent versioned storage.
`sh 'mvn deploy'`	Runs mvn deploy — builds AND uploads to the repository in pom.xml distributionManagement.
`stage('Deploy to Tomcat')`	Fourth stage: copy the .war to Tomcat webapps/ and trigger hot-deployment.
`deploy adapters: [...]`	Uses the "Deploy to Container" Jenkins plugin to push .war to Tomcat via the Manager API.
`post { success / failure }`	Runs AFTER all stages. success{} only if all passed. failure{} if any stage failed. Used for notifications, cleanup.

Complete Jenkinsfile

```
pipeline { agent any tools { maven 'Maven' } stages { stage('Checkout') { steps { checkout scm } } stage('Build') { steps { sh 'mvn clean package' } } stage('Push to Nexus') { steps { sh 'mvn deploy' } } stage('Deploy to Tomcat') { steps { deploy adapters:[tomcat9(credentialsId:'tomcat-creds', url:'http://localhost:8082')], contextPath:'hello-app', war:'target/*.war' } } } post { success { echo 'Deployed successfully!' } failure { echo 'Pipeline failed - check logs' } } }
```

05 Interview Q&A; — Concepts

25 questions you will be asked

CI/CD & General Pipeline

Q1. What is CI/CD?

CI (Continuous Integration) means every code push automatically triggers a build and test run, catching integration issues early. CD (Continuous Delivery/Deployment) extends this by automatically deploying every successful build to an environment. Together they eliminate manual build and deploy steps and allow teams to ship software faster with confidence.

Q2. Explain your CI/CD pipeline end to end.

Code is pushed to GitHub. Jenkins detects the push via webhook. Jenkins pulls the code, runs mvn clean package (Maven compiles and tests), produces a .war artifact. Jenkins pushes the .war to Nexus for versioned storage. Jenkins deploys the .war to Tomcat, which hot-deploys it. The entire flow is automatic — no human steps after the git push.

Q3. What is a webhook and why is it used?

A webhook is an HTTP callback — GitHub sends a POST request to Jenkins at http://:8080/github-webhook/ whenever code is pushed. Jenkins receives this signal and immediately triggers the pipeline. Without a webhook, Jenkins would need to poll GitHub every few minutes which is inefficient.

Q4. What is Pipeline as Code?

Pipeline as Code means the pipeline definition (Jenkinsfile) lives inside the source code repository alongside the application code. This means pipeline changes are version-controlled, reviewable via pull requests, and reproducible. The alternative — configuring the pipeline in the Jenkins UI — is fragile and not auditable.

Q5. What happens when a Jenkins build fails?

The pipeline stops at the failed stage. Subsequent stages are skipped. Jenkins marks the build red in the UI. The console output log shows exactly where it failed and the error message. I would read the console output first, identify whether it is a compilation error, test failure, or connectivity issue, fix the root cause, and push the fix to GitHub which triggers a new build.

Maven

Q1. What does mvn clean package do?

It runs two Maven goals: "clean" deletes the target/ folder (removes previous build output), and "package" executes the build lifecycle: validate -> compile (.java to .class) -> test (runs unit tests) -> package (bundles compiled code and resources into a .war or .jar). The result is a deployable artifact in the target/ directory.

Q2. What is pom.xml and what does it contain?

pom.xml (Project Object Model) is the Maven project configuration file. It defines: groupId (organization namespace), artifactId (project name), version (current version), packaging (war/jar), dependencies (external libraries with versions and scopes), build plugins, and distributionManagement (where to deploy artifacts).

Q3. What does provided mean?

It tells Maven this dependency is needed for compilation but should NOT be bundled in the final .war, because the runtime environment (Tomcat) already provides it. Without this, the servlet-api.jar would be inside the .war and Tomcat also has one — two copies cause ClassLoader conflicts and runtime errors.

Q4. What is the difference between mvn package and mvn deploy?

mvn package compiles, tests, and bundles the code into a .war/jar locally in target/. mvn deploy does everything package does PLUS uploads the artifact to the remote repository defined in in pom.xml (e.g., Nexus). Deploy is used in CI/CD pipelines to version and store artifacts centrally.

Q5. What is a .war file?

WAR stands for Web Application aRchive. It is a zip-format package containing compiled .class files (NOT source .java files), dependency JARs (those not marked provided), WEB-INF/web.xml or servlet annotations, and static resources. It requires a servlet container like Tomcat to run. It cannot execute by itself.

05 Interview Q&A; — Continued

Tomcat, Nexus, Jenkins

Tomcat

Q1. What is Tomcat and why is it needed?

Tomcat is a servlet container — it provides the HTTP listening layer and servlet lifecycle management that a .war file needs to serve web requests. A .war is inert by itself: it has no mechanism to listen on a port or handle HTTP. Tomcat listens on port 8082, receives requests, finds the matching servlet class, calls doGet()/doPost(), and returns the response.

Q2. What is the Tomcat webapps/ folder?

It is the deployment folder. When you copy a .war file into webapps/, Tomcat detects it automatically and deploys it — no restart needed. Tomcat also "explodes" (unzips) the .war into a folder of the same name which is what it actually serves from. When deploying a new version, remove the old .war AND the old folder, then copy the new .war.

Q3. Where do you look when a Tomcat deployment fails?

The first place is /opt/tomcat9/logs/catalina.out — Tomcat writes all deployment events, ClassNotFoundExceptions, port conflicts, and startup errors there. Run: tail -f /opt/tomcat9/logs/catalina.out while deploying to see events in real time.

Q4. What is the difference between Tomcat 9 and Tomcat 10?

Tomcat 9 uses the javax.servlet.* package namespace (Java EE). Tomcat 10+ migrated to jakarta.servlet.* (Jakarta EE). If you deploy a javax-based .war to Tomcat 10, the servlet will silently fail to load. Always match your servlet API import namespace to your Tomcat version.

Nexus

Q1. What is Nexus and why is it needed?

Nexus is an artifact repository manager. It stores every built .war/.jar with a permanent version number and timestamp. Without Nexus, you have no history of what was built and deployed — no rollback capability, no audit trail. Nexus gives every artifact a unique URL so any machine (Jenkins, another server, a developer) can download a specific version.

Q2. What is the difference between maven-releases and maven-snapshots?

maven-releases stores final, immutable versions (1.0.0, 1.0.1, 2.0.0). Once pushed, a release version cannot be overwritten — it is permanent. maven-snapshots stores work-in-progress versions (1.0.0-SNAPSHOT) which CAN be overwritten. Use snapshots during development, releases when the code is ready to ship.

Q3. How would you roll back a deployment using Nexus?

Go to Nexus, find the previous good version (e.g., hello-app-1.0.2.war), copy its direct URL. In Jenkins, trigger a deployment with that specific version — or manually copy the old .war from Nexus to Tomcat webapps/. This is exactly why artifact storage exists: every version is retrievable at any time.

Q4. What are the checksums (.md5, .sha1) stored alongside artifacts in Nexus?

They are integrity verification files. After downloading an artifact, Maven computes its checksum and compares it against the .md5/.sha1 file from Nexus. If they match, the download is intact. If not, the file was corrupted in transit. This prevents deploying a corrupted binary.

Jenkins

Q1. What is Jenkins?

Jenkins is an open-source automation server that runs CI/CD pipelines. It installs as a system service (systemctl), runs on port 8080, and reads Jenkinsfiles from Git repositories. When triggered (manually or by webhook), it executes the pipeline stages on its build agents and records results, logs, and artifacts for every build.

Q2. What is a Jenkinsfile?

A text file named "Jenkinsfile" at the root of your Git repository that defines the entire CI/CD pipeline in Groovy DSL (Declarative Pipeline syntax). Because it lives in the repo, pipeline changes are version-controlled and reviewable. Jenkins reads it when the pipeline runs.

Q3. What is the Jenkins workspace?

A folder on the Jenkins server (at /var/lib/jenkins/workspace//) where Jenkins clones the repository and runs all build commands. Every build starts with a fresh clone. The .war produced by Maven lives here at target/ until Jenkins pushes it to Nexus or Tomcat.

Q4. What is the difference between a Freestyle Job and a Pipeline Job in Jenkins?

A Freestyle Job is configured entirely through the Jenkins UI — build steps, triggers, notifications are all point-and-click. It is inflexible and not version-controlled. A Pipeline Job reads its definition from a Jenkinsfile in the Git repo — it is code, reviewable, reproducible, and supports complex multi-stage flows. Pipeline is the modern approach.

Q5. What plugins are essential for this CI/CD stack?

Git Plugin (clone from GitHub), Maven Integration (run Maven goals), Pipeline Plugin (read Jenkinsfiles), Deploy to Container Plugin (deploy to Tomcat via Manager API), Credentials Plugin (store Nexus and Tomcat passwords securely), GitHub Plugin (receive webhooks from GitHub).

Q6. How does Jenkins manage credentials securely?

Jenkins stores credentials in an encrypted store at /var/lib/jenkins/secrets/. In the UI: Manage Jenkins -> Credentials -> Add Credentials. Credentials are referenced by an ID in the Jenkinsfile (e.g., credentialsId: "tomcat-creds") and never appear as plain text in logs or config files.

06 Scenario-Based Questions

Walk me through your thinking — what interviewers really test

Scenario 1: Your Jenkins build is failing. Walk me through how you debug it.

1. Open Jenkins UI, click the failed build, click Console Output. Read the last 20 lines — the error is there. 2. Identify which stage failed: Checkout (Git issue), Build (Maven/Java error), Push to Nexus (connectivity/auth issue), or Deploy (Tomcat issue). 3. For Maven errors: read the [ERROR] line — it shows the file and line number. 4. For Nexus push failures: check ~/.m2/settings.xml credentials and Nexus URL. 5. For Tomcat deploy failures: check catalina.out. Fix root cause, push code, new build triggers automatically.

Scenario 2: You deployed a bad version and the app is broken. How do you roll back?

1. Go to Nexus UI -> Browse -> maven-releases. Find the last good version (e.g., 1.0.2). 2. Copy its direct download URL. 3. On EC2: wget that URL to download the old .war. 4. Remove current .war and exploded folder from Tomcat webapps/. 5. Copy old .war to webapps/. Tomcat hot-deploys in seconds. This is exactly why Nexus exists — every version is permanently stored and retrievable.

Scenario 3: Two services are fighting over port 8080. What happens and how do you fix it?

The second service to start gets "Address already in use: 8080" and crashes. Only one process can bind to a port at a time. Fix: change one service's port. For Tomcat: edit /opt/tomcat9/conf/server.xml, change . For Jenkins: edit /etc/default/jenkins, change HTTP_PORT=8080 to another port. We moved Tomcat to 8082 specifically because Jenkins owns 8080.

Scenario 4: A developer asks why their code works locally but fails the Jenkins build. What do you check?

1. Java version mismatch: local JDK vs Jenkins JDK. 2. Maven version difference. 3. Missing environment variables (API keys, config) present locally but not on Jenkins server. 4. Dependencies cached locally but not in Nexus or Maven Central. 5. Branch mismatch: Jenkins is building a different branch. 6. Test failures: tests pass locally due to local state but fail on clean Jenkins workspace. The clean Jenkins workspace is the feature, not the bug — it enforces reproducibility.

Scenario 5: How would you handle different environments (Dev, QA, Production) in this pipeline?

Create separate pipeline stages or branches for each environment. Use different Jenkinsfiles or parameterized builds. Store environment-specific config in Jenkins credentials or environment variables — never hardcode in code. For example: build once, store artifact in Nexus, deploy the same .war to Dev Tomcat first, run tests, then deploy same artifact to QA, test again, then Production. Same binary, different environments — ensures what you tested is what you deploy.

07 Key Concepts & Glossary

One-line definitions for every term you will encounter

Artifact
The compiled, packaged output of a build. In this project: the .war file. Artifacts are versioned, stored in Nexus, and deployed to Tomcat.

Servlet
A Java class that handles HTTP requests. Extends HttpServlet and overrides doGet()/doPost(). Your HelloServlet is a servlet. Tomcat manages servlet lifecycles.

EBS (Elastic Block Store)
AWS virtual hard disk attached to your EC2 instance. Where all your files actually live. Persists when you stop the instance. Deleted by default when you terminate.

Build Lifecycle
Maven's ordered sequence of phases: validate -> compile -> test -> package -> verify -> install -> deploy. Each phase includes the ones before it.

Hot Deployment
Tomcat's ability to detect a new .war in webapps/ and deploy it automatically without restart. Drop the file and Tomcat picks it up within seconds.

Context Path
The URL path for a deployed app on Tomcat. Derived from the .war filename: hello-app-1.0.1.war -> context path /hello-app-1.0.1. Full URL: http://ip:8082/hello-app-1.0.1/hello.

Declarative Pipeline
Jenkins Jenkinsfile syntax using pipeline{}, stages{}, stage{}, steps{} blocks. Readable, structured, recommended over Scripted Pipeline.

SCM (Source Code Management)
Git/GitHub in this project. Jenkins uses SCM to know where to clone code from. "checkout scm" in Jenkinsfile uses the repo configured in the job.

systemctl
Linux service manager. Used to start/stop/enable Jenkins as a persistent system service. "enable" means auto-start on reboot. "status" shows if running.

distributionManagement
pom.xml section that tells mvn deploy where to push artifacts. Contains the Nexus repository URL and repository ID (must match settings.xml server id).

Credentials (Jenkins)
Securely stored username/password or tokens in Jenkins. Referenced by ID in Jenkinsfile. Never appear in plain text in logs.

Build Agent
The machine Jenkins runs pipeline stages on. In this project, the Jenkins server itself is the only agent (agent any). In large teams, multiple dedicated agents run builds in parallel.

Webhook
HTTP POST callback GitHub sends to Jenkins on code push. Configured at: GitHub repo -> Settings -> Webhooks -> http://:8080/github-webhook/

Convention over Configuration
Maven's design philosophy. If you follow the standard folder structure (src/main/java/, src/test/java/, etc.), Maven needs zero configuration to find your code. Deviating requires explicit configuration.

Rollback Reverting to a previous working version. Made possible by Nexus: every version is stored permanently and can be redeployed to Tomcat at any time.	

08 Commands Cheat Sheet

Every command from Sessions 1-6

MAVEN

```
# Build mvn clean package mvn clean deploy mvn compile mvn test # Check mvn -version ls -lh target/ unzip -l target/hello-app-1.0.1.war
```

TOMCAT

```
# Lifecycle sudo /opt/tomcat9/bin/startup.sh sudo /opt/tomcat9/bin/shutdown.sh # Deploy sudo cp hello-app-1.0.1.war /opt/tomcat9/webapps/ sudo rm -rf /opt/tomcat9/webapps/hello-app-1.0.0* # Debug tail -f /opt/tomcat9/logs/catalina.out ls /opt/tomcat9/webapps/
```

NEXUS

```
# Lifecycle sudo -u nexus /opt/nexus/bin/nexus start sudo -u nexus /opt/nexus/bin/nexus stop sudo -u nexus /opt/nexus/bin/nexus status # Debug tail -f /opt/sonatype-work/nexus3/log/nexus.log cat /opt/sonatype-work/nexus3/admin.password
```

JENKINS

```
# Service sudo systemctl start jenkins sudo systemctl stop jenkins sudo systemctl restart jenkins sudo systemctl status jenkins sudo systemctl enable jenkins # Debug tail -f /var/log/jenkins/jenkins.log sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

LINUX / EC2

```
# System free -h # check RAM df -h # check disk lsblk # list block devices ps aux | grep tomcat # find processes # SSH ssh -i devops-lab.pem ubuntu@ sudo netstat -tlnp | grep 8080 # check port
```

GIT / GITHUB

```
# Push code git init && git add . git commit -m "message" git remote add origin git branch -M main git push -u origin main # Update git add . && git commit -m "update" git push
```

09 Interview Strategy

How to think and what to say

The One-Minute Pipeline Answer

Memorize this. Deliver it with confidence. This is your answer to "Tell me about a CI/CD pipeline you have built."

> *"The pipeline starts when a developer pushes code to GitHub. Jenkins detects the push via a webhook and pulls the latest code automatically. Jenkins then runs Maven — mvn clean package — which compiles the Java code, runs tests, and produces a .war artifact. Jenkins pushes that artifact to Nexus, our artifact repository, where it is versioned permanently and available for rollback. Finally Jenkins deploys the .war to Tomcat — the Java servlet container — which hot-deploys it and serves it over HTTP. The entire flow is automatic from push to deployment with no human intervention."*

What Interviewers Are Really Testing

Do you understand why each tool exists?
Not just "Maven builds" but WHY Maven vs writing build scripts manually. Not just "Nexus stores" but WHY artifact storage vs re-building every time.

Can you debug a broken pipeline?
They will describe a failure scenario. Walk through your debugging process: which log file, which stage, what you look for. Show systematic thinking.

Do you know the difference between the tools?
Jenkins vs GitHub Actions, Nexus vs S3, Tomcat vs Nginx. Knowing where each fits and when to use which shows depth beyond tutorial knowledge.

Can you talk about real concerns?
Security (credentials in Jenkins, not in code), cost (stop EC2 when not using), scalability (one EC2 is fine for learning, not for production).

Are you honest about experience level?
Say: "I built this as a learning project to understand CI/CD fundamentals." Do not claim production experience you don't have. Interviewers probe and the lie collapses.

Common Mistakes to Avoid in Interviews

Mistake	What to say instead

Saying "I know Jenkins" broadly	Interviewers will ask follow-up questions about plugin configs, credential management, log files. If you cannot answer specifics, say: "I understand pipeline architecture and have worked with Jenkins for this learning project."
Confusing Tomcat and Apache HTTP Server	Tomcat is a servlet container for Java apps. Apache HTTP Server serves static files. They are different tools. A common interview trick: "what is the difference?"
Saying "tail -f" when asked where logs are	tail -f is a command, not a location. The answer is the file path: /opt/tomcat9/logs/catalina.out or /var/log/jenkins/jenkins.log. Then mention tail -f as how you read it live.
Claiming the pipeline is "production-ready"	It is a learning setup: no HTTPS, no auth on Tomcat, no HA, no monitoring. Be honest: "This is a learning environment. Production would add SSL, proper RBAC, high availability, and monitoring."